

CC4030

February 22, 2026

1 4.3 Uniendo todos los componentes

En este punto ya estamos en posibilidad de ensamblar las partes de todo el sistema creativo y generar resultados.

En las siguientes celdas iremos integrando cada una de las partes de todo el sistema, reutilizando gran parte del código que revisamos previamente e introduciendo algunos elementos necesarios para la implementación del algoritmo genético.

2 Integración

1. Importamos todos los módulos y funciones que necesitaremos

```
[30]: from pintura_abstracta import discretizar_figuras, adhesion_a_composicion, \
      ↪ saturacion_promedio, \
normalizar_calidades, tchebycheff_ponderado, mpa, cruza_uniforme, \
      ↪ mutacion_uniforme, reemplazo_elitista, \
PROPORCION_AUREA, PLANTILLA_COMPOSICION_ROMBOIDAL, \
      ↪ escalar_plantilla_a_proporcion_aurea, \
poblacion_aleatoria, novedad, Hex6, genera_cuadrantes
```

2. A partir del código que utilizamos previamente, vamos a crear diversas funciones que servirán para ensamblar el algoritmo genético completo.

- 2.1. Creamos una función integrada para evaluar una pintura.

Mejora de eficiencia a la evaluación: Como habíamos mencionado anteriormente, es posible anular criterios simplemente asignándoles un peso de 0, la anulación se hace efectiva al momento de que se combinan los criterios en la función de combinación ya que el valor que se haya calculado para la evaluación de un criterio en particular será 0 al multiplicarse por su peso 0, sin embargo, esto no implica que el cálculo de la evaluación para ese criterio no se vaya a realizar. Se puede hacer el proceso de evaluación un poco más eficiente si omitimos por completo el cálculo del valor de evaluación para un criterio cuando su peso sea 0:

```
[31]: def evaluar(pintura_i, poblacion_historica, historico_calidades, \
      ↪ rangos_calidad, pesos, calidad_utopica,
```

```

        patron_composicional, cuadrantes, archivo):

    # Determina el valor de composición
    calidad_criterio1_composicion = adhesion_a_composicion(pintura_i,
    ↪patron_composicional, archivo) if pesos[0] > 0 else 0

    # Determina el valor de saturación de color
    calidad_criterio2_saturacion = saturacion_promedio(pintura_i, archivo) if
    ↪pesos[1] > 0 else 0

    # Determina el valor de novedad
    calidad_criterio3_novedad = novedad(pintura_i, cuadrantes, archivo,
    ↪poblacion_historica, historico_calidades) \
        if pesos[2] > 0 else 0

    # Integra las evaluaciones
    calidades = [calidad_criterio1_composicion, calidad_criterio2_saturacion,
    ↪calidad_criterio3_novedad]

    # Normaliza las evaluaciones
    calidades_normalizadas = normalizar_calidades(calidades, rangos_calidad)

    # Normaliza la calidad utópica
    calidad_utopica_normalizada = normalizar_calidades(calidad_utopica,
    ↪rangos_calidad)

    # Determina la calidad global
    calidad_global = -tchebycheff_ponderado(calidades_normalizadas, pesos,
    ↪calidad_utopica_normalizada)
    return calidad_global

```

2.2. Establecemos la función (muestreo proporcional a la aptitud) para seleccionar 2 pinturas a

[32]: seleccionar_ascendencia = mpa

2.3. Establecemos la función para combinar dos pinturas ascendentes y generar dos recombinantes

[33]: recombinar = cruza_uniforme

2.4. Establecemos la función para mutar (mutación uniforme) un recombinante y generar una pintura

[34]: mutar = mutacion_uniforme

2.5. Definimos una función de criterio de paro que hará que la evolución se detenga cuando se l

```
[35]: def detener_evolucion(generacion_actual, maximo):  
    detener = generacion_actual >= maximo  
    return detener
```

2.6. Establecemos una función más que se encargará de realizar el reemplazo de la generación a

```
[36]: reemplazar_generacion = reemplazo_elitista
```

3. Inicializamos las dimensiones del lienzo, la representación interna del mismo y el parámetro que controla la cantidad de figuras que contendrá cada pintura.

```
[37]: ancho_lienzo = 600  
alto_lienzo = int(ancho_lienzo / PROPORCION_AUREA)  
lienzo = {'ancho': ancho_lienzo, 'alto': alto_lienzo}  
  
num_figuras = 30
```

4. Inicializamos los rangos de valores de las variables que conforman la representación interna y los colocamos en una lista.

```
[38]: # Rangos de variables para la representación interna de una figura  
rango_tipo_figura = (1, 3)  
rango_posicion_x = (0, lienzo['ancho'])  
rango_posicion_y = (0, lienzo['alto'])  
rango_ancho = (5, int(lienzo['ancho'] / 4))  
rango_alto = (5, int(lienzo['alto'] / 4))  
rango_orientacion = (0, 359)  
rango_color = (Hex6(0x000000), Hex6(0xFFFFFF))  
rangos = [rango_tipo_figura,  
          rango_posicion_x,  
          rango_posicion_y,  
          rango_ancho,  
          rango_alto,  
          rango_orientacion,  
          rango_color]
```

5. Inicializamos los rangos de evaluación de los criterios de calidad y los colocamos en una lista.

```
[39]: # Creación del patrón composicional  
plantilla = PLANTILLA_COMPOSICION_ROMBOIDAL  
patron_composicional = escalar_plantilla_a_proporcion_aurea(lienzo, plantilla)  
  
# Definición del rango de evaluación para el criterio de composición  
rango_composicion = (0, 1)
```

```
# Definición del rango de evaluación para el criterio de saturación de color
rango_saturacion = (0, 1)

# Definición del rango de evaluación para el criterio de novedad
rango_novedad = (0, 1)

# Integración de los rangos de evaluación
rangos_calidad = [rango_composicion, rango_saturacion, rango_novedad]
```

6. Inicializamos las dimensiones de la retícula para los cuadrantes que se utilizarán para medir la novedad y generamos los cuadrantes.

```
[40]: renglones = 2
      columnas = 3
      dimensiones_reticula = (renglones, columnas)
      cuadrantes = genera_cuadrantes(lienzo, dimensiones_reticula)
```

7. Inicializamos el archivo o historial en donde se almacenarán las mediciones para la determinación de las medidas estéticas y de novedad.

```
[41]: archivo = {}
```

8. Establecemos los pesos o ponderaciones para cada criterio de calidad requeridos por la función de combinación de criterios. Para este ejercicio se han establecido los siguientes valores: 53.2% para la composición, 27.5% para la saturación de color y 19.3% para la novedad.

```
[42]: pesos = (0.532, 0.275, 0.193)
```

9. Definimos los valores de calidad (normalizados de origen) para nuestra pintura de referencia utópica o ideal que servirá para calcular la calidad global de los miembros de nuestra población de pinturas.

```
[43]: calidad_utopica = (1, 1, 1)
```

10. Inicializamos los parámetros del algoritmo genético:

- `tamano_poblacion`, tamaño de la población (cantidad de pinturas).
- `num_generaciones`, número de generaciones (iteraciones) que nuestra población de pinturas evolucionará.
- `tamano_elitismo`, cantidad de pinturas (las mejores evaluadas) que se pasarán de una generación a la siguiente.
- `tamano_ascendencia`, cantidad de pinturas que deben ser seleccionadas cada vez que se vaya a realizar una recombinación.
- `tamano_descendencia`, cantidad mínima de pinturas que deben generarse por recombinación y mutación a partir de una ascendencia seleccionada.

```
[44]: tamaño_poblacion = 8
      num_generaciones = 80
      tamaño_ascendencia = 2
      tamaño_elitismo = tamaño_poblacion // 2
      tamaño_descendencia = tamaño_poblacion - tamaño_elitismo
```

11. Inicializamos los parámetros para la creación de descendencia:

- Probabilidad de intercambio (probabilidad de una pintura “hija” de heredar figuras de uno de sus ascendentes).
- Probabilidad de mutación (probabilidad de que alguna figura de una pintura recombinante que heredó de su ascendencia sea reemplazada por una figura aleatoria).

```
[45]: probabilidad_intercambio = 0.5
      probabilidad_mutacion = 0.045
```

12. Inicializamos el contador de generaciones contra el cual se comparará el criterio de paro de la evolución.

```
[46]: g = 1
```

17. Creamos nuestra población inicial a partir del código utilizado previamente e inicializamos una lista donde almacenaremos las evaluaciones de calidad.

Para fines de poder explorar los resultados de nuestro sistema, implementaremos la población y las evaluaciones de calidad de una manera que mantengan un histórico, es decir que sea posible acceder a cualquier pintura de cualquier generación y a su correspondiente evaluación combinada.

```
[47]: poblacion = []
      poblacion.append(poblacion_aleatoria(tamaño_poblacion, num_figuras, rangos))
      calidades = []
```

13. Finalmente, ensamblamos las funciones creadas en un algoritmo genético que evolucionará nuestra población inicial de pinturas abstractas. De acuerdo con el funcionamiento de un algoritmo genético, definiremos un proceso evolutivo continuo que se detendrá únicamente cuando la función de criterio de paro lo indique, mientras esto no suceda, la población será evaluada, se seleccionaran ascendentes, se creará descendencia (a través de recombinación y mutación) y se reemplazará la generación actual con la siguiente.

El tiempo de ejecución del algoritmo genético puede impactarse (en mayor o menor grado) debido a factores como: tamaño de la población, cantidad de figuras por pintura, número de generaciones, entre otros.

```
[48]: while True:
      # Evaluación
```

```

        calidades.append([evaluar(pintura, poblacion, calidades, rangos_calidad,
→pesos, calidad_utopica,
                                patron_composicional, cuadrantes, archivo)
                            for pintura in poblacion[-1]])
    # Revisión del criterio de paro
    if detener_evolucion(g, num_generaciones):
        break
    else:
        # Creación de descendencia
        descendencia_potencial = []
        # ¿Hasta que la cantidad de hijos potenciales sea mayor o igual que la
→cantidad de nuevas pinturas
        # necesarias para la siguiente generación, es decir, restando al tamaño
→de la población, el tamaño
        # del elitismo utilizado.
        while len(descendencia_potencial) < tamaño_descendencia:
            # Selección de la ascendencia (2 padres)
            ascendente1, ascendente2 = seleccionar_ascendencia(poblacion[-1],
→calidades[-1], tamaño_ascendencia)
            # Generación de 2 nuevas pinturas por recombinación de la
→ascendencia
            recombinante1, recombinante2 = recombinar(ascendente1, ascendente2,
→probabilidad_intercambio)
            # Mutación de las 2 nuevas pinturas
            mutantes = [mutar(recombinante1, probabilidad_mutacion, rangos),
                        mutar(recombinante2, probabilidad_mutacion, rangos)]
            # Acumulado de los hijos potenciales
            descendencia_potencial = descendencia_potencial + mutantes
            # Reemplazo de la generación actual con la siguiente generación
            poblacion.append(reemplazar_generacion(poblacion[-1], calidades[-1],
                                                    descendencia_potencial,
→tamaño_elitismo))
            # Incremento del contador de generaciones
            g = g + 1

```

En la siguiente celda utilizamos la función de conveniencia `pintar_mejor_y_peor_evaluadas` para desplegar en pantalla los resultados más distintivos del proceso evolutivo. Es posible especificar en esta función un patrón composicional como parámetro `patron` para que este patrón se dibuje (como X grises) encima de la pintura generada para poder observar el grado en que la composición de la pintura se aproxima o no al patrón establecido.

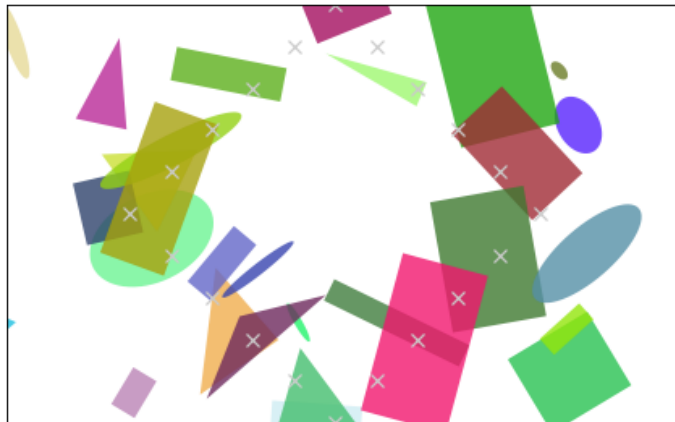
```

[49]: from pintura_abstracta import pintar_mejor_y_peor_evaluadas
      pintar_mejor_y_peor_evaluadas(poblacion, calidades, lienzo,
→patron=patron_composicional)

```



Peor evaluada:((2, 44, 197, 32, 16, 261, 0x3571E3), ..., (1, 275, 162, 80, 30, 294, 0xD8BBB6))



Mejor evaluada:((1, 424, 20, 134, 89, 300, 0xABDA10), ..., (1, 194, 107, 80, 10, 128, 0x3744AF))

14. Un paso adicional es encapsular el algoritmo en una función para poderlo reutilizar más fácilmente en algún experimento posterior. Existen estrategias más avanzadas y robustas para generalizar y parametrizar este tipo de algoritmos (Ej.: para su uso en otros dominios o tipos de artefacto) pero de momento bastará con lo siguiente:

```
[50]: def evolucionar(num_generaciones, tamaño_poblacion, num_figuras, rangos,
                    rangos_calidad, pesos, calidad_utopica, patron_composicional,
                    cuadrantes, archivo,
                    probabilidad_intercambio, probabilidad_mutacion,
                    tamaño_ascendencia, tamaño_descendencia, tamaño_elitismo):
    # Inicialización del contador de generaciones
    g = 1
```

```

# Generación de la población inicial
poblacion = []
poblacion.append(poblacion_aleatoria(tamano_poblacion, num_figuras,
→rangos))
calidades = []
while True:
    # Evaluación
    calidades.append([evaluar(pintura, poblacion, calidades,
→rangos_calidad, pesos, calidad_utopica,
                                patron_composicional, cuadrantes, archivo)
                    for pintura in poblacion[-1]])
    # Revisión del criterio de paro
    if detener_evolucion(g, num_generaciones):
        break
    else:
        # Creación de descendencia
        descendencia_potencial = []
        # Hasta que la cantidad de hijos potenciales sea mayor o igual que
→la cantidad de nuevas pinturas
        # necesarias para la siguiente generación, es decir, restando al
→tamaño de la población, el tamaño
        # del elitismo utilizado.
        while len(descendencia_potencial) < tamano_descendencia:
            # Selección de la ascendencia (2 padres)
            ascendente1, ascendente2 =
→seleccionar_ascendencia(poblacion[-1], calidades[-1], tamano_ascendencia)
            # Generación de 2 nuevas pinturas por recombinación de la
→ascendencia
            recombinante1, recombinante2 = recombinar(ascendente1,
→ascendente2, probabilidad_intercambio)
            # Mutación de las 2 nuevas pinturas
            mutantes = [mutar(recombinante, probabilidad_mutacion, rangos)
                        for recombinante in [recombinante1, recombinante2]]
            # Acumulado de los hijos potenciales
            descendencia_potencial = descendencia_potencial + mutantes
            # Reemplazo de la generación actual con la siguiente generación
            poblacion.append(reemplazar_generacion(poblacion[-1], calidades[-1],
                                                    descendencia_potencial,
→tamano_elitismo))
            # Incremento del contador de generaciones
            g = g + 1
        return poblacion, calidades

```

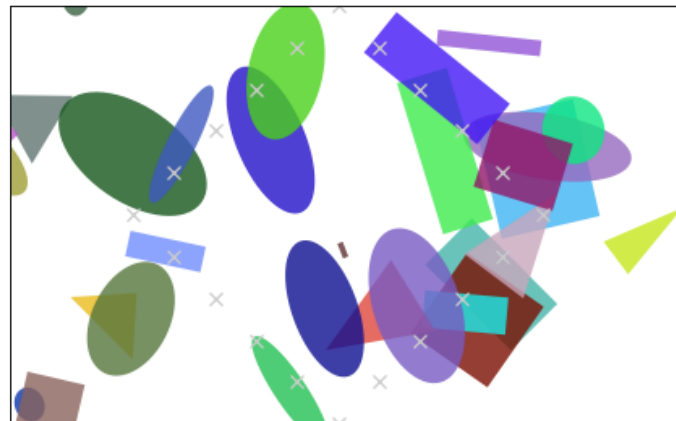
De esta manera podemos ejecutar nuestro sistema creativo con una sola llamada enviando todos los parámetros necesarios.

Recuerda siempre inicializar el archivo del historial para la evaluación de novedad antes de una ejecución.

```
[51]: archivo = {}
poblacion, calidades = evolucionar(num_generaciones, tamaño_poblacion,
    ↳ num_figuras, rangos,
                                rangos_calidad, pesos, calidad_utopica,
    ↳ patron_composicional,
                                cuadrantes, archivo,
                                probabilidad_intercambio,
    ↳ probabilidad_mutacion,
                                tamaño_ascendencia, tamaño_descendencia,
    ↳ tamaño_elitismo)
pintar_mejor_y_peor_evaluadas(poblacion, calidades, lienzo,
    ↳ patron=patron_composicional)
```



Peor evaluada:((1, 400, 255, 120, 65, 24, 0x1D3F46), ..., (3, 112, 208, 145, 19, 54, 0x2CA0BA))

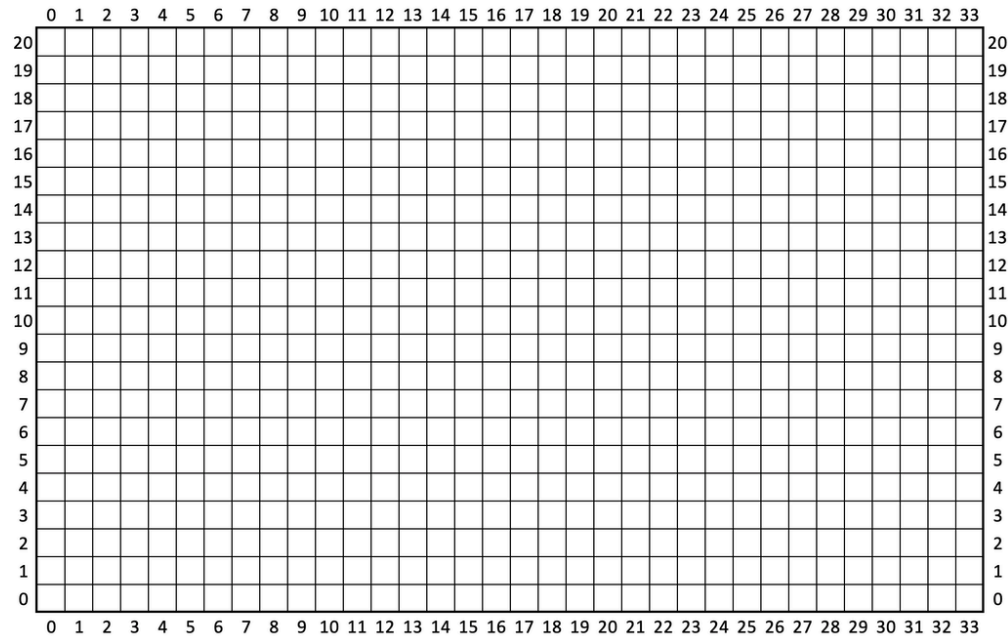


Mejor evaluada:((3, 526, 161, 36, 74, 37, 0xC6E81A), ..., (1, 264, 245, 123, 66, 168, 0x4AD11A))

3 Ejercicio 1:

A continuación, vas a inventar tu propio patrón composicional que servirá para evaluar las pinturas que el sistema creativo genere, para ello puedes auxiliarte de la retícula aurea:

Descarga la imagen de la retícula y marca los puntos que desees que formen parte de tu plantilla. Trata de que los puntos no sean demasiados (quizá entre 10 y 20), el tiempo de evaluación es proporcional al número de puntos de referencia.



Retícula para plantilla de patrón composicional

Crea la lista de puntos (x,y) y guárdalos en una variable llamada `plantilla_patron_composicional`.

Observa los resultados de la evolución utilizando tu propio patrón composicional.

Si desees que se despliegue tu patrón como guía sobre la pintura generada, asigna el patrón escalado `patron_composicional` al parámetro `patron` de la función `pintar_mejor_evaluada`, de lo contrario déjalo con el valor `None` o simplemente elimina el parámetro.

```
[52]: from pintura_abstracta import pintar_mejor_evaluada

plantilla_patron_composicional = [(9, 18), (7, 17), (6, 15), (7, 13), (9, 12),
→ (12, 12), (15, 12), (18, 11),
(19, 9), (18, 7), (16, 5), (15, 3), (16, 1),
→ (18, 0), (21, 0), (25, 2),
(28, 4), (31, 6)]
```

```

patron_composicional = escalar_plantilla_a_proporcion_aurea(lienzo,
↳ plantilla_patron_composicional)

#Reinicializar el historial
archivo = {}

poblacion, calidades = evolucionar(num_generaciones, tamaño_poblacion,
↳ num_figuras, rangos,
                                rangos_calidad, pesos, calidad_utopica,
↳ patron_composicional,
                                cuadrantes, archivo,
                                probabilidad_intercambio,
↳ probabilidad_mutacion,
                                tamaño_ascendencia, tamaño_descendencia,
↳ tamaño_elitismo)

pintar_mejor_evaluada(poblacion, calidades, lienzo, patron=patron_composicional)

```



Mejor evaluada:((2, 226, 90, 105, 76, 11, 0x16D908), ..., (2, 312, 18, 124, 49, 32, 0xCF5117))

4 Ejercicio 2:

Como se había mencionado anteriormente, es posible anular criterios con el fin de experimentar y explorar los resultados que el sistema creativo obtenga. En las siguientes celdas experimentarás anulando uno u otro criterio, además de invertir criterios individuales a través de la evaluación ideal o utópica.

Si es necesario aumenta poco a poco el número de generaciones para obtener resultados más notorios. Si estableces un número excesivamente grande puedes ocasionar que tu libreta Jupyter quede bloqueada.

4.0.1 Solo criterio 1: Composición (máxima adhesión)

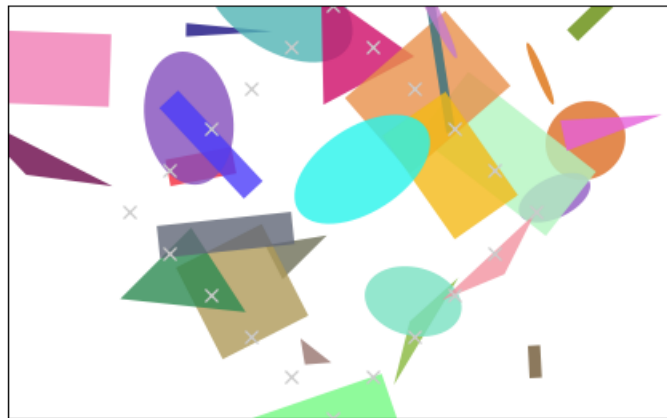
Para generar pinturas únicamente considerando el criterio de composición debes anular los otros dos criterios de evaluación mediante los pesos además de asegurarte de asignar en la calidad utópica, un valor igual o mayor al límite superior del rango para el criterio de composición:

```
[53]: plantilla_patron_composicional = PLANTILLA_COMPOSICION_ROMBOIDAL
patron_composicional = escalar_plantilla_a_proporcion_aurea(lienzo,
    ↪plantilla_patron_composicional)

num_generaciones = 80

pesos = (1, 0, 0)
calidad_utopica = (1, 1, 1)
archivo = {}
poblacion, calidades = evolucionar(num_generaciones, tamaño_poblacion,
    ↪num_figuras, rangos,
                                rangos_calidad, pesos, calidad_utopica,
    ↪patron_composicional,
                                cuadrantes, archivo,
                                probabilidad_intercambio,
    ↪probabilidad_mutacion,
                                tamaño_ascendencia, tamaño_descendencia,
    ↪tamaño_elitismo)

pintar_mejor_evaluada(poblacion, calidades, lienzo, patron=patron_composicional)
```



Mejor evaluada:((2, 146, 208, 60, 24, 101, 0xF4293C), ..., (1, 407, 325, 130, 11, 206, 0xC77BCD))

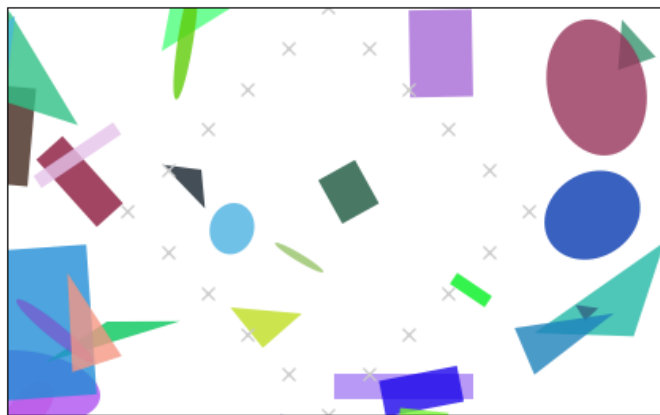
4.0.2 Solo criterio 1: Composición (mínima adhesión)

Continuando únicamente con el criterio de composición puedes invertir el efecto del criterio estableciendo como valor utópico un valor igual o menor al valor mínimo de ese criterio.

```
[54]: # Reestablece el número de generaciones en caso de que se haya modificado en la
      ↪actividad anterior.
num_generaciones = 80

pesos = (1, 0, 0)
calidad_utopica = (0, 1, 1)
archivo = {}
poblacion, calidades = evolucionar(num_generaciones, tamano_poblacion,
      ↪num_figuras, rangos,
                                rangos_calidad, pesos, calidad_utopica,
      ↪patron_composicional,
                                cuadrantes, archivo,
                                probabilidad_intercambio,
      ↪probabilidad_mutacion,
                                tamano_ascendencia, tamano_descendencia,
      ↪tamano_elitismo)

pintar_mejor_evaluada(poblacion, calidades, lienzo, patron=patron_composicional)
```



Mejor evaluada:((3, 596, 157, 144, 54, 305, 0x22BAA6), ..., (1, 160, 285, 110, 14, 171, 0x5FCC07))

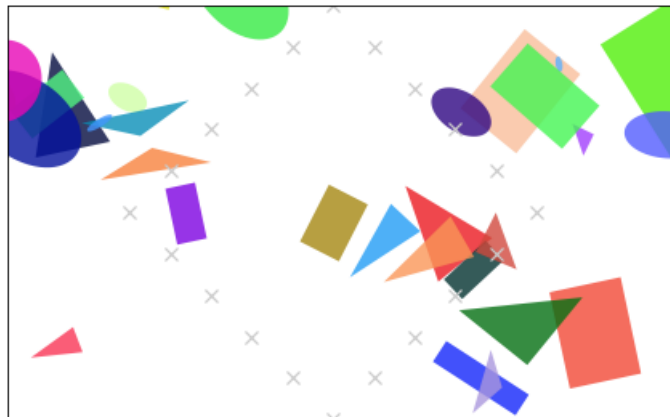
4.0.3 Solo criterio 2: Saturación de color (saturación máxima)

Ahora deberás dejar habilitado únicamente al criterio de saturación de color y utilizar el valor máximo para la calidad utópica.

```
[55]: # Reestablece el número de generaciones en caso de que se haya modificado en la
      ↪actividad anterior.
num_generaciones = 80

pesos = (0, 1, 0)
calidad_utopica = (1, 1, 1)
archivo = {}
poblacion, calidades = evolucionar(num_generaciones, tamaño_poblacion,
      ↪num_figuras, rangos,
                                rangos_calidad, pesos, calidad_utopica,
      ↪patron_composicional,
                                cuadrantes, archivo,
                                probabilidad_intercambio,
      ↪probabilidad_mutacion,
                                tamaño_ascendencia, tamaño_descendencia,
      ↪tamaño_elitismo)

pintar_mejor_evaluada(poblacion, calidades, lienzo, patron=patron_composicional)
```



Mejor evaluada:((1, 129, 289, 37, 23, 242, 0xD0FEA6), ..., (3, 514, 109, 111, 55, 276, 0x047012))

4.0.4 Solo criterio 2: Saturación de color (saturación mínima)

Utiliza el valor mínimo para este criterio como valor utópico.

Observa qué pasa cuando incrementas más el número de generaciones permitiéndole al algoritmo generar pinturas cada vez más acorde al criterio de evaluación. Nota: En este criterio puedes aumentar más la cantidad de generaciones ya que es el criterio menos costoso.

```
[56]: # Reestablece el número de generaciones en caso de que se haya modificado en la
      ↪actividad anterior.
```

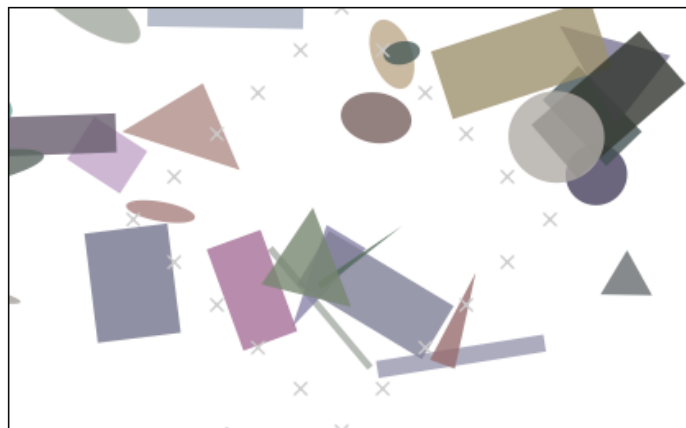
```

num_generaciones = 2000

pesos = (0, 1, 0)
calidad_utopica = (1, 0, 1)
archivo = {}
poblacion, calidades = evolucionar(num_generaciones, tamaño_poblacion,
    ↪ num_figuras, rangos,
                                rangos_calidad, pesos, calidad_utopica,
    ↪ patron_composicional,
                                cuadrantes, archivo,
                                probabilidad_intercambio,
    ↪ probabilidad_mutacion,
                                tamaño_ascendencia, tamaño_descendencia,
    ↪ tamaño_elitismo)

pintar_mejor_evaluada(poblacion, calidades, lienzo, patron=patron_composicional)

```



Mejor evaluada:((3, 518, 119, 45, 39, 89, 0x696D70), ..., (3, 100, 261, 108, 62, 72, 0xB18D87))

4.0.5 Solo criterio 3: Novedad (máxima novedad)

Utiliza la función `pintar_mejor_evaluada_por_generacion` para pintar las pinturas mejor evaluadas de un grupo de generaciones.

Observa cómo la novedad es cada vez menos notoria conforme progresa la evolución a través de las generaciones, esto puede representar la situación de que cuanto mayor sea la cantidad de artefactos distintos de una clase, mayor la dificultad de innovar.

```
[57]: from pintura_abstracta import pintar_mejor_evaluada_por_generacion
```

```

# Reestablece el número de generaciones en caso de que se haya modificado en la
→actividad anterior.
num_generaciones = 120

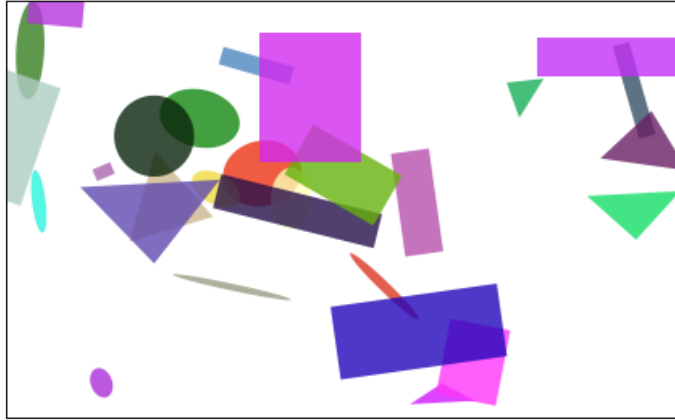
pesos = (0, 0, 1)
calidad_utopica = (1, 1, 1)
archivo = {}
poblacion, calidades = evolucionar(num_generaciones, tamaño_poblacion,
→num_figuras, rangos,
                                rangos_calidad, pesos, calidad_utopica,
→patron_composicional,
                                cuadrantes, archivo,
                                probabilidad_intercambio,
→probabilidad_mutacion,
                                tamaño_ascendencia, tamaño_descendencia,
→tamaño_elitismo)

pintar_mejor_evaluada_por_generacion(poblacion, calidades, lienzo, desde=1,
→hasta=4, patron=None)
pintar_mejor_evaluada_por_generacion(poblacion, calidades, lienzo,
→desde=num_generaciones - 3, patron=None)

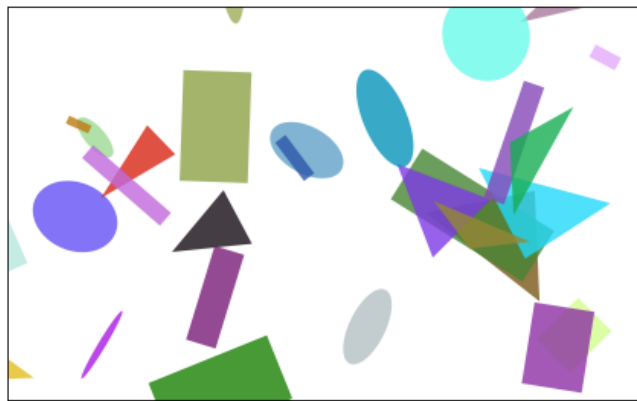
```



Mejor evaluada (1):((1, 361, 151, 79, 68, 217, 0x814203), ..., (2, 347, 68, 88, 83, 256, 0xEE434A))



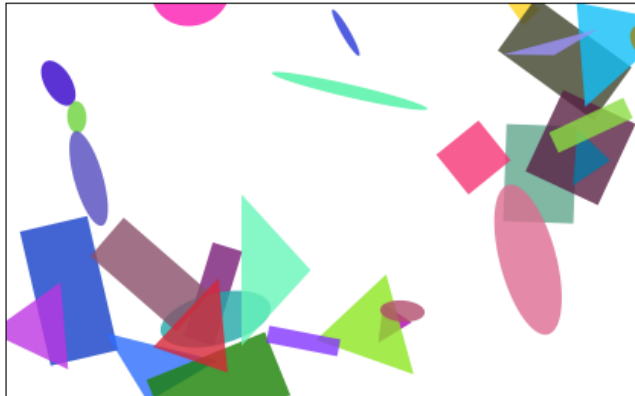
Mejor evaluada (2):((2, 539, 329, 85, 15, 16, 0x355066), ..., (3, 528, 230, 78, 48, 82, 0x691F64))



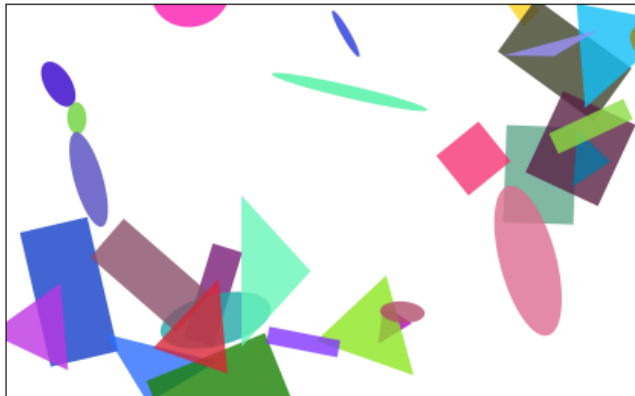
Mejor evaluada (3):((3, 482, 355, 106, 30, 103, 0xA87A9C), ..., (1, 228, 355, 131, 26, 185, 0xA0AB59))



Mejor evaluada (117):((2, 445, 70, 80, 92, 219, 0xE396A7), ..., (3, 175, 16, 50, 48, 98, 0x82AA89))



Mejor evaluada (118):((3, 487, 349, 21, 84, 133, 0xFFD21D), ..., (2, 520, 229, 77, 21, 115, 0x8EDC45))



Mejor evaluada (119):((3, 487, 349, 21, 84, 133, 0xFFD21D), ..., (2, 520, 229, 77, 21, 115, 0x8EDC45))

4.0.6 Solo criterio 3: Novedad (nula novedad)

Solo es necesario que ejecutes el proceso por 2 generaciones, observa como las mejores pinturas son iguales, esto es porque estamos favoreciendo la nula novedad.

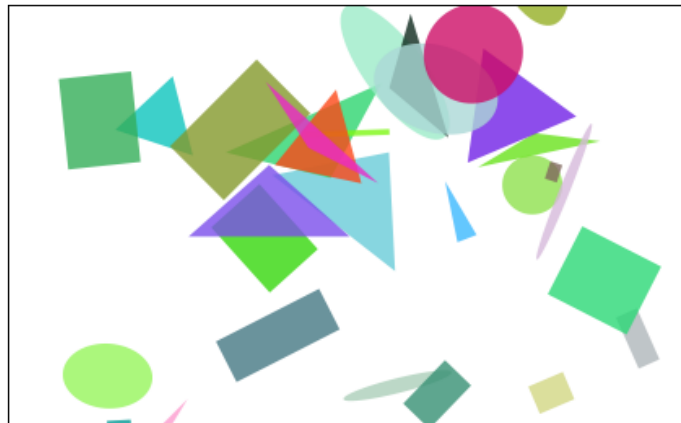
```
[58]: num_generaciones = 2
      pesos = (0, 0, 1)
      calidad_utopica = (1, 1, 0)
      archivo = {}
      poblacion, calidades = evolucionar(num_generaciones, tamano_poblacion,
      ↪num_figuras, rangos,
```

```

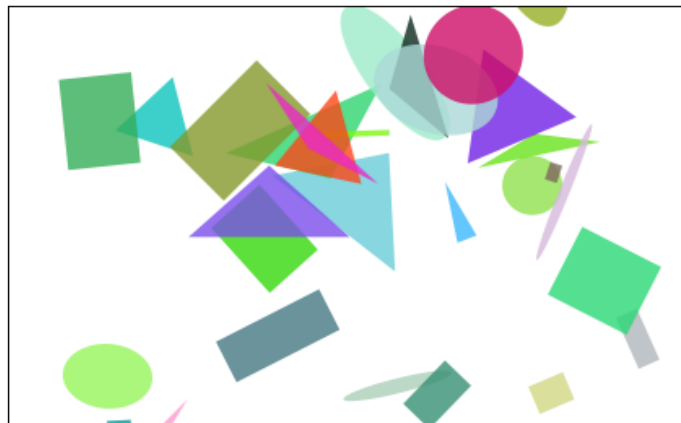
rangos_calidad, pesos, calidad_utopica,
→patron_composicional,
cuadrantes, archivo,
probabilidad_intercambio,
→probabilidad_mutacion,
tamaño_ascendencia, tamaño_descendencia,
→tamaño_elitismo)

pintar_mejor_evaluada_por_generacion(poblacion, calidades, lienzo, hasta=2,
→patron=None)

```



Mejor evaluada (0):((2, 108, 5, 21, 91, 273, 0x029A95), ..., (2, 473, 217, 11, 16, 72, 0x80685D))



Mejor evaluada (1):((2, 108, 5, 21, 91, 273, 0x029A95), ..., (2, 473, 217, 11, 16, 72, 0x80685D))

[]: